Dictionary_Assigment_ChinmayeeNayak

In [1]:
```
# Q1:Initialize a dictionary "emp_info" with below details
# In - emp_info['Tom']
# Out - {'email':'tom_latham019@gmail.com', 'Phone': +1987654321, 'City': 'Californ

# In - emp_info['Kathy']
# Out - {'email':'kathy_abram897@gmail.com', 'Phone': +1887654321, 'City': 'New Yor
```

In [2]:
```
# Solution
emp_info = {
    'Tom': {'email': 'tom_latham019@gmail.com', 'Phone': +1987654321, 'City': 'Cali
    'Kathy': {'email': 'kathy_abram897@gmail.com', 'Phone': +1887654321, 'City': 'N
}
```

In [3]:
```
# Accessing Tom's information
tom_info = emp_info['Tom']
print(tom_info)  # Output will be: {'email': 'tom_latham019@gmail.com', 'Phone': 19

# Accessing Kathy's information
kathy_info = emp_info['Kathy']
print(kathy_info)  # Output will be: {'email': 'kathy_abram897@gmail.com', 'Phone':
```

```
{'email': 'tom_latham019@gmail.com', 'Phone': 1987654321, 'City': 'California'}
{'email': 'kathy_abram897@gmail.com', 'Phone': 1887654321, 'City': 'New York'}
```

In [4]:
```
#Q2: Create a dictionary out of below inputs
# lst1 = ['emp1', 'emp2', 'emp3']
# emp_key = ['e_name', 'e_id', 'e_sal']
# emp1_val = ['John', 'SG101', '$10,000']
# emp2_val = ['Smith', 'SG102', '$9,000']
# emp3_val = ['Peter', 'SG103', '$9,500']

# Expected Output:- {'emp1':{'e_name':'John', 'e_id':'SG101', 'e_sal':$10,000},
#                    'emp2':{'e_name':'Smith', 'e_id':'SG102', 'e_sal':$9,000},
#                    'emp3':{'e_name':'Peter', 'e_id':'SG103', 'e_sal':$9,500}}
```

In [12]:
```
# Solution
lst1 = ['emp1', 'emp2', 'emp3']
emp_key = ['e_name', 'e_id', 'e_sal']
emp1_val = ['John', 'SG101', '$10,000']
emp2_val = ['Smith', 'SG102', '$9,000']
emp3_val = ['Peter', 'SG103', '$9,500']

employee_dict = {}

# Loop through the employee IDs
for i in range(len(lst1)):
    employee_info = {}
    for j in range(len(emp_key)):
        employee_info[emp_key[j]] = emp1_val[j] if i == 0 else (emp2_val[j] if i ==
    employee_dict[lst1[i]] = employee_info
```

```
print(employee_dict)
```

```
{'emp1': {'e_name': 'John', 'e_id': 'SG101', 'e_sal': '$10,000'}, 'emp2': {'e_name':
'Smith', 'e_id': 'SG102', 'e_sal': '$9,000'}, 'emp3': {'e_name': 'Peter', 'e_id': 'S
G103', 'e_sal': '$9,500'}}
```

In [13]:
```
#Q3: Acess the value of key 'history'

sampleDict = {
   "class":{
      "student":{
         "name":"Mike",
         "marks":{
            "physics":70,
            "history":80
         }
      }
   }
}
```

In [14]:
```
# Solution
# Given dictionary
sampleDict = {
    "class": {
        "student": {
            "name": "Mike",
            "marks": {
                "physics": 70,
                "history": 80
            }
        }
    }
}

# Access the value of the 'history' key
history_mark = sampleDict["class"]["student"]["marks"]["history"]

# Print the value
print("History Mark:", history_mark)
```

```
History Mark: 80
```

In [15]:
```
#Q4: Initialize dictionary with default values. Inputs are:-
# employees = ['Kelly', 'Emma', 'John']
# defaults = {"designation": 'Application Developer', "salary": 8000}

#Expected output:- {'Kelly': {'designation': 'Application Developer', 'salary': 800
#                    'Emma': {'designation': 'Application Developer', 'salary': 8000
#                    'John': {'designation': 'Application Developer', 'salary': 8000
```

In [16]:
```
# Solution
employees = ['Kelly', 'Emma', 'John']
defaults = {"designation": 'Application Developer', "salary": 8000}

employee_info = {employee: defaults.copy() for employee in employees}
```

```
print(employee_info)
```

```
{'Kelly': {'designation': 'Application Developer', 'salary': 8000}, 'Emma': {'design
ation': 'Application Developer', 'salary': 8000}, 'John': {'designation': 'Applicati
on Developer', 'salary': 8000}}
```

OR

In [17]:
```
employees = ['Kelly', 'Emma', 'John']
defaults = {"designation": 'Application Developer', "salary": 8000}

employee_info = {}
for employee in employees:
    employee_info[employee] = defaults.copy()

print(employee_info)
```

```
{'Kelly': {'designation': 'Application Developer', 'salary': 8000}, 'Emma': {'design
ation': 'Application Developer', 'salary': 8000}, 'John': {'designation': 'Applicati
on Developer', 'salary': 8000}}
```

In [18]:
```
# Q5: In gene expression, mRNA is transcribed from a DNA template.
# The 4 nucleotide bases of A, T, C, G corresponds to the U, A, G, C bases of the m
# Write a function that returns the mRNA transcript given the sequence of a DNA str

# Use a dictionary to provide the mapping of DNA to RNA bases.#
```

In [19]:
```
# Solution
def transcribe_dna_to_mrna(dna_sequence):
    # Create a dictionary to map DNA bases to RNA bases
    dna_to_rna = {'A': 'U', 'T': 'A', 'C': 'G', 'G': 'C'}

    # Initialize an empty mRNA sequence
    mrna_sequence = ""

    # Iterate through the DNA sequence and transcribe each base
    for base in dna_sequence:
        if base in dna_to_rna:
            mrna_sequence += dna_to_rna[base]
        else:
            # Handle invalid input (e.g., if non-standard bases are present)
            return "Invalid DNA sequence"

    return mrna_sequence

# Example usage
dna_sequence = "ATCG"
mrna_transcript = transcribe_dna_to_mrna(dna_sequence)
print("mRNA Transcript:", mrna_transcript)
```

```
mRNA Transcript: UAGC
```

OR

In [20]:
```python
def transcribe_dna_to_mrna(dna_sequence):
    # Create a dictionary to map DNA bases to RNA bases
    dna_to_rna = {'A': 'U', 'T': 'A', 'C': 'G', 'G': 'C'}

    # Use a list comprehension to transcribe the DNA sequence
    mrna_sequence = [dna_to_rna[base] for base in dna_sequence if base in dna_to_rn

    # Convert the list to a string
    mrna_sequence = ''.join(mrna_sequence)

    # Handle invalid input
    if len(mrna_sequence) != len(dna_sequence):
        return "Invalid DNA sequence"

    return mrna_sequence

# Example usage
dna_sequence = "ATCG"
mrna_transcript = transcribe_dna_to_mrna(dna_sequence)
print("mRNA Transcript:", mrna_transcript)
```

```
mRNA Transcript: UAGC
```

In [21]:
```python
#Q6: Write a function which takes a word as input and returns a dictionary with let
# In - count_letter('google.com')
# Out - {'g': 2, 'o': 3, 'l': 1, 'e': 1, '.': 1, 'c': 1, 'm': 1}
```

In [23]:
```python
def count_letter(word):
    letter_count = {}  # Initialize an empty dictionary to store the letter counts

    for letter in word:
        if letter in letter_count:
            letter_count[letter] += 1
        else:
            letter_count[letter] = 1

    return letter_count

# Example usage here the input as "google.com"
word = "google.com"
result = count_letter(word)
print(result)

# It will give the output as {'g': 2, 'o': 3, 'l': 1, 'e': 1, '.': 1, 'c': 1, 'm':
```

```
{'g': 2, 'o': 3, 'l': 1, 'e': 1, '.': 1, 'c': 1, 'm': 1}
```

In [24]:
```
#Q7: A DNA strand consisting of the 4 nucleotide bases is usually represented with
# Write a function that computes the base composition of a given DNA sequence.

# In - baseComposition("CTATCGGCACCCTTTCAGCA")
# Out - {'A': 4, 'C': 8, 'T': 5,  'G': 3 }

# In - baseComposition("AGT")
# Out - {'A': 1, 'C': 0, 'T': 1,  'G': 1 }
```

In [25]:
```
# Solution
def baseComposition(dna_sequence):
    base_count = {'A': 0, 'C': 0, 'T': 0, 'G': 0}  # Initialize a dictionary to sto

    for base in dna_sequence:
        if base in base_count:
            base_count[base] += 1

    return base_count

# Example usage Input as baseComposition("CTATCGGCACCCTTTCAGCA") & Input as baseCom
sequence1 = "CTATCGGCACCCTTTCAGCA"
result1 = baseComposition(sequence1)
print(result1)  # Output: {'A': 4, 'C': 8, 'T': 5, 'G': 3}

sequence2 = "AGT"
result2 = baseComposition(sequence2)
print(result2)  # Output: {'A': 1, 'C': 0, 'T': 1, 'G': 1}
```

```
{'A': 4, 'C': 8, 'T': 5, 'G': 3}
{'A': 1, 'C': 0, 'T': 1, 'G': 1}
```

In [26]:
```
#Q8: [MCQ] Suppose "d" is an empty dictionary, which statement does not assign "d"
# 1. d = {"Name": "Tom" }
# 2. d["Name"] = "Tom"
# 3. d.update({"Name": "Tom" })
# 4. d.setdefault("Name", "Tom")
# 5. None of the above.
```

In [31]:
```
# Solution
#The statement that does not assign d with {"Name":"Tom"} is:
#d["Name"] = "Tom"

# Initial empty dictionary
d = {}

# Option 1
d = {"Name": "Tom"}
print("Option 1:", d)  # Output: {'Name': 'Tom'}

# Reset the dictionary to empty
d = {}

# Option 2
d["Name"] = "Tom"
print("Option 2:", d)  # Output: {'Name': 'Tom'}
```

```
# Reset the dictionary to empty
d = {}

# Option 3
d.update({"Name": "Tom"})
print("Option 3:", d)  # Output: {'Name': 'Tom'}

# Reset the dictionary to empty
d = {}

# Option 4
d.setdefault("Name", "Tom")
print("Option 4:", d)  # Output: {'Name': 'Tom'}
```

```
Option 1: {'Name': 'Tom'}
Option 2: {'Name': 'Tom'}
Option 3: {'Name': 'Tom'}
Option 4: {'Name': 'Tom'}
```

Explantaion of this code d = {"Name": "Tom" }: This directly assigns the dictionary {"Name": "Tom"} to the variable d, so d becomes {"Name": "Tom"}. d["Name"] = "Tom": This statement updates the existing dictionary d by adding a key-value pair, so d becomes {"Name": "Tom"}. If d was initially an empty dictionary, it would create a new key-value pair. d.update({"Name": "Tom" }): This statement updates the existing dictionary d with the key-value pair from the provided dictionary, so d becomes {"Name": "Tom"}. If d was initially an empty dictionary, it would create a new dictionary with the key-value pair. d.setdefault("Name", "Tom"): This statement is similar to option 2, and it sets the value of the key "Name" in the existing dictionary d to "Tom". If the key "Name" doesn't exist in d, it will create the key-value pair. So, d becomes {"Name": "Tom"}. Therefore, the correct answer is option 2

In [32]:
```
#Q9: [MCQ] d = {"a":1, "b":2}. Which of the statements returns [1,2]?
# 1. d.keys()
# 2. d.values()
# 3. d.items()
# 4. d.popitem()
# 5. None of the above.
```

Solution #Explanation d = {"a":1, "b":2} The statement that returns [1,2] is: d.values() Here's an explanation for each option: d.keys(): This returns a list-like view of the dictionary's keys, so it would return ['a', 'b']. d.values(): This returns a list-like view of the dictionary's values, so it would return [1, 2], which is [1, 2]. d.items(): This returns a list-like view of the dictionary's key-value pairs, so it would return [['a', 1], ['b', 2]]. d.popitem(): This removes and returns an arbitrary key-value pair from the dictionary as a tuple. It doesn't return a list, so it wouldn't return [1, 2]. Therefore, the correct answer is option 2.

In [33]:
```
#Q10: [MCQ] Which of the following declarations is not valid for 'dict' type?
# 1. d = {"Name": "Tom" }
# 2. d = { (1,3,4): 4.5 }
# 3. d = { ["First", "Last"]: (1,3) }
# 4. d = { 1: 0.4 }
# 5. None of the above
```

Solution # Explanation The declaration that is not valid for a 'dict' type in Python is: d = { ["First", "Last"]: (1, 3) } In Python, dictionary keys must be hashable and immutable. In option 3, a list ["First", "Last"] is used as a key, and lists are mutable, which is why this declaration is not valid. The other options use strings, tuples, or numbers as keys, all of which are hashable and therefore valid for a dictionary.

In [34]:
```
# Q11:Write a function reverseLookup(dictionary, value) that takes in a dictionary
# and a value as arguments and returns a sorted list of all keys that contains the
# The function will return an empty list if no match is found.

# In - reverseLookup({'a':1, 'b':2, 'c':2}, 1)
# Out - ['a']
```

```
# In - reverseLookup({'a':1, 'b':2, 'c':2}, 2)
# Out - ['b', 'c']
# In - reverseLookup({'a':1, 'b':2, 'c':2}, 3)
# Out - []
```

In [36]:
```
# Solution
def reverseLookup(dictionary, value):
    result = []

    for key, val in dictionary.items():
        if val == value:
            result.append(key)

    result.sort()  # Sort the list of keys

    return result

# Example usage:
dict1 = {'a': 1, 'b': 2, 'c': 2}
value1 = 1
print(reverseLookup(dict1, value1))  # Output: ['a']

dict2 = {'a': 1, 'b': 2, 'c': 2}
value2 = 2
print(reverseLookup(dict2, value2))  # Output: ['b', 'c']

dict3 = {'a': 1, 'b': 2, 'c': 2}
value3 = 3
print(reverseLookup(dict3, value3))  # Output: []
```

```
['a']
['b', 'c']
[]
```

In [37]:
```
#Q12: Write a function invertDictionary(d) that takes in a dictionary as argument a
# In - invertDictionary({'a':1, 'b':2, 'c':3, 'd':2})
# Out - {1: ['a'], 2: ['b', 'd'], 3: ['c']}
# In - invertDictionary({'a':3, 'b':3, 'c':3})
# Out - {3: ['a', 'c', 'b']}
# In - invertDictionary({'a':2, 'b':1, 'c':2, 'd':1})
# Out - {1: ['b', 'd'], 2: ['a', 'c']}
```

In [43]:
```
# Solution
def invertDictionary(d):
    inverted_dict = {}

    for key, value in d.items():
        if value not in inverted_dict:
            inverted_dict[value] = [key]
        else:
            inverted_dict[value].append(key)

    sorted_inverted_dict = {key: sorted(inverted_dict[key], key=lambda x: (x != 'a'

    return sorted_inverted_dict
```

```python
# Example usage:
dict1 = {'a': 1, 'b': 2, 'c': 3, 'd': 2}
result1 = invertDictionary(dict1)
print(result1)  # Output: {1: ['a'], 2: ['b', 'd'], 3: ['c']}

dict2 = {'a': 3, 'b': 3, 'c': 3}
result2 = invertDictionary(dict2)
print(result2)  # Output: {3: ['a', 'c', 'b']}

dict3 = {'a': 2, 'b': 1, 'c': 2, 'd': 1}
result3 = invertDictionary(dict3)
print(result3)  # Output: {1: ['b', 'd'], 2: ['a', 'c']}
```

```
{1: ['a'], 2: ['b', 'd'], 3: ['c']}
{3: ['a', 'c', 'b']}
{1: ['b', 'd'], 2: ['a', 'c']}
```

In [44]:

```python
#Q13: Write a function that converts a sparse vector into a dictionary as described
# In - convertVector([1, 0, 0, 2, 0, 0, 0, 3, 0, 0, 0, 0, 4])
# Out - {0: 1, 3: 2, 7: 3, 12: 4}
# In - convertVector([1, 0, 1 , 0, 2, 0, 1, 0, 0, 1, 0])
# Out - {0: 1, 2: 1, 4: 2, 6: 1, 9: 1}
# In - convertVector([0, 0, 0, 0, 0])
# Out - {}
```

In [45]:

```python
# Solution

def convertVector(vector):
    sparse_dict = {}
    for i, value in enumerate(vector):
        if value != 0:
            sparse_dict[i] = value
    return sparse_dict

# Example usage:
vector1 = [1, 0, 0, 2, 0, 0, 0, 3, 0, 0, 0, 0, 4]
result1 = convertVector(vector1)
print(result1)  # Output: {0: 1, 3: 2, 7: 3, 12: 4}

vector2 = [1, 0, 1, 0, 2, 0, 1, 0, 0, 1, 0]
result2 = convertVector(vector2)
print(result2)  # Output: {0: 1, 2: 1, 4: 2, 6: 1, 9: 1}

vector3 = [0, 0, 0, 0, 0]
result3 = convertVector(vector3)
print(result3)  # Output: {}
```

```
{0: 1, 3: 2, 7: 3, 12: 4}
{0: 1, 2: 1, 4: 2, 6: 1, 9: 1}
{}
```

In [46]:

```python
#Q14: Write a function that converts a dictionary back to its sparse vector represe
# In - convertDictionary({0: 1, 3: 2, 7: 3, 12: 4})
# Out - [1, 0, 0, 2, 0, 0, 0, 3, 0, 0, 0, 0, 4]
# In - convertDictionary({0: 1, 2: 1, 4: 2, 6: 1, 9: 1})
# Out - [1, 0, 1, 0, 2, 0, 1, 0, 0, 1]
```

```
# In - convertDictionary({})
# Out - []
```

In [47]:
```
def convertDictionary(sparse_dict):
    if not sparse_dict:
        return []

    max_index = max(sparse_dict.keys())
    vector = [0] * (max_index + 1)

    for key, value in sparse_dict.items():
        vector[key] = value

    return vector

# Example usage:
dict1 = {0: 1, 3: 2, 7: 3, 12: 4}
result1 = convertDictionary(dict1)
print(result1)  # Output: [1, 0, 0, 2, 0, 0, 0, 3, 0, 0, 0, 0, 4]

dict2 = {0: 1, 2: 1, 4: 2, 6: 1, 9: 1}
result2 = convertDictionary(dict2)
print(result2)  # Output: [1, 0, 1, 0, 2, 0, 1, 0, 0, 1]

dict3 = {}
result3 = convertDictionary(dict3)
print(result3)  # Output: []
```

```
[1, 0, 0, 2, 0, 0, 0, 3, 0, 0, 0, 0, 4]
[1, 0, 1, 0, 2, 0, 1, 0, 0, 1]
[]
```

In [48]:
```
#Q15: Given a Python dictionary, Change Brad’s salary to 8500
# sampleDict = {
#      'emp1': {'name': 'Jhon', 'salary': 7500},
#      'emp2': {'name': 'Emma', 'salary': 8000},
#      'emp3': {'name': 'Brad', 'salary': 6500}
# }

# Expected Output
# sampleDict = {
#      'emp1': {'name': 'Jhon', 'salary': 7500},
#      'emp2': {'name': 'Emma', 'salary': 8000},
#      'emp3': {'name': 'Brad', 'salary': 8500}
# }
```

In [49]:
```
# Solution

sampleDict = {
    'emp1': {'name': 'Jhon', 'salary': 7500},
    'emp2': {'name': 'Emma', 'salary': 8000},
    'emp3': {'name': 'Brad', 'salary': 6500}
}

# Change Brad's salary to 8500
sampleDict['emp3']['salary'] = 8500
```

```
# Print the updated sampleDict
print(sampleDict)
```

```
{'emp1': {'name': 'Jhon', 'salary': 7500}, 'emp2': {'name': 'Emma', 'salary': 8000},
'emp3': {'name': 'Brad', 'salary': 8500}}
```

In [50]:
```
#Q16: Get the key corresponding to the minimum value from the following dictionary
# sampleDict = {
#   'Physics': 82,
#   'Math': 65,
#   'history': 75
# }

# Expected Output
# Math
```

In [52]:
```
# Solution
sampleDict = {
    'Physics': 82,
    'Math': 65,
    'history': 75
}

min_key = min(sampleDict, key=sampleDict.get)

print(min_key)
```

```
Math
```

In [53]:
```
#Q17: Rename key city to location in the following dictionary
# sampleDict = {
#   "name": "Kelly",
#   "age":25,
#   "salary": 8000,
#   "city": "New york"
# }

# Expected Output
# {
#   "name": "Kelly",
#   "age":25,
#   "salary": 8000,
#   "location": "New york"
# }
```

In [54]:
```
# Solution

sampleDict = {
    "name": "Kelly",
    "age": 25,
    "salary": 8000,
    "city": "New York"
}

# Rename the key "city" to "location"
```

```python
sampleDict["location"] = sampleDict.pop("city")

# Print the updated sampleDict
print(sampleDict)
```

```
{'name': 'Kelly', 'age': 25, 'salary': 8000, 'location': 'New York'}
```

In [55]:
```python
#Q18: Check if a value 200 exists in a dictionary
# sampleDict = {'a': 100, 'b': 200, 'c': 300}

# Expected Output: True
```

In [57]:
```python
# Solution

sampleDict = {'a': 100, 'b': 200, 'c': 300}

# Check if the value 200 exists in the dictionary
if 200 in sampleDict.values():
    print(True)
else:
    print(False)
```

```
True
```

In [58]:
```python
#Q19: Delete set of keys from Python Dictionary
# sampleDict = {
#   "name": "Kelly",
#   "age":25,
#   "salary": 8000,
#   "city": "New york"

# }
# keysToRemove = ["name", "salary"]

# Expected Output:
# {'city': 'New york', 'age': 25}
```

In [67]:
```python
# Solution
sampleDict = {
    "name": "Kelly",
    "age": 25,
    "salary": 8000,
    "city": "New York"
}

keysToRemove = ["name", "salary"]

# Create a new dictionary by filtering out the keys to be removed
resultDict = {key: sampleDict[key] for key in sampleDict if key not in keysToRemove

# Rearrange the keys in the desired order
resultDict = {"city": resultDict["city"], "age": resultDict["age"]}

# Print the updated resultDict
print(resultDict)
```

```
{'city': 'New York', 'age': 25}
```

In [ ]: